

Contents

Ultimate Amiga OS — DOS & Scripting Manual	1
Table of Contents	1
Introduction	1
Getting Started	2
Shell Built-In Commands	2
C: Binaries (Native Commands)	4
Scripting & Flow Control	11
Environment Variables	13
File Assigns	14
I/O Redirection	14
Resident Commands	15
Example Scripts	15
Quick Reference Card	16

Ultimate Amiga OS — DOS & Scripting Manual

Version 0.1.0-dev | Ultimate Amiga OS Shell Reference

Table of Contents

1. Introduction
 2. Getting Started
 3. Shell Built-In Commands
 4. C: Binaries (Native Commands)
 5. Ring-3 Userspace Commands
 6. Scripting & Flow Control
 7. Environment Variables
 8. File Assigns
 9. I/O Redirection
 10. Resident Commands
 11. Example Scripts
 12. Quick Reference Card
-

Introduction

The UAOS shell is a hybrid AmigaDOS-style command interpreter that runs natively on the x86_64 kernel. It combines traditional AmigaDOS conventions (assigns, C:, S:, ENV: paths) with modern bare-metal capabilities including TCP/IP networking, block-device management, and script flow control.

Commands are case-insensitive. Both `DIR` and `dir` work identically.

Command Types

Type	Description	Examples
Shell Built-Ins	Executed directly by the shell; manage shell state	<code>help</code> , <code>cd</code> , <code>alias</code> , <code>set</code> , <code>path</code>
C: Binaries	Native x86_64 kernel commands registered in <code>native_cmd.c</code>	<code>dir</code> , <code>copy</code> , <code>ping</code> , <code>loadwb</code>

Type	Description	Examples
Ring-3 Userspace M68k Binaries	Native x86-64 ELF64 binaries using INT 0x80 Legacy Amiga Hunk binaries run via emulation	<code>pwd</code> , <code>file</code> , <code>strings</code> , <code>find</code> , <code>Guide</code> any file found in <code>PATH</code>

Command Templates

Native commands may declare an AmigaDOS-style template for automatic argument parsing:

Qualifier	Meaning	Example
/A	Required argument	FILE/A
/K	Keyword argument	FROM/K
/S	Switch (boolean)	ALL/S
/N	Numeric	COL/K/N
/M	Multiple values	BUTTON/M
/F	Free-form (absorbs rest)	STRING/F

Example: `copy FROM/A,TO/A,ALL/S,CLONE/S` means `copy` requires `FROM` and `TO`, and accepts `ALL` and `CLONE` switches.

Getting Started

The Prompt

When you open a Shell window you see:

```
UAOS>
```

Type commands and press **Enter** to execute.

Command History

- **Up Arrow** — recall previous command
- **Down Arrow** — recall next command
- **Tab** — auto-complete filenames and command names

Stopping a Command

There is no dedicated break key. Commands that loop (e.g. `ping`) run a fixed number of iterations and yield automatically to keep the desktop responsive.

Shell Built-In Commands

These commands are handled directly by the shell and are not separate binaries on `C:`.

`help`

Show the complete list of built-ins and `C:` binaries.

```
UAOS> help
```

cd [path]

Change working directory, or print the current directory if no argument is given.

```
UAOS> cd Workbench:C
UAOS> cd RAM:T
UAOS> cd ..          ; parent directory
UAOS> cd             ; prints current directory
```

alias [name cmd]

Create, update, or list command aliases.

```
UAOS> alias ll "dir"
UAOS> alias      ; list all aliases
UAOS> alias h "help"
```

unalias <name>

Remove an alias.

```
UAOS> unalias ll
```

set [name value]

Set or list **local** environment variables (per-shell, not persisted).

```
UAOS> set greeting "Hello World"
UAOS> set           ; list all local variables
```

unset <name>

Remove a local environment variable.

```
UAOS> unset greeting
```

path [dirs...]

Show or set the command search path. The path is a space-separated list of directories searched when a command is not a built-in or native binary.

```
UAOS> path          ; show current path
UAOS> path C: S:    ; set path to C: and S:
```

setenv <name> <value>

Set a **global** environment variable. Writes to both the local shell store and **ENV:<name>** so it persists across sessions.

```
UAOS> setenv Language "english"
```

unsetenv <name>

Remove a global environment variable from both the local store and **ENV:.**

```
UAOS> unsetenv Language
```

showconfig

Show hardware configuration in Amiga-style format (processor, RAM, boards).

```
UAOS> showconfig
```

C: Binaries (Native Commands)

All commands below are native x86_64 binaries located conceptually on C:. They are dispatched directly by the kernel without M68k emulation.

System Information

version Display OS version and hardware summary.

```
UAOS> version
Ultimate Amiga OS  v0.1.0-dev
Kernel: x86_64 ELF64, Multiboot2, long mode
Display: 1024x768 32bpp linear framebuffer
Input: PS/2 keyboard + mouse, IRQ1/IRQ12
```

mem Display memory layout.

```
UAOS> mem
RAM: 512 MB (QEMU)
Kernel load: 0x0000000000100000
Framebuffer: mapped (GOP physical address)
Stack: 16 KB (bootstrap), no heap allocator yet
```

libs List all loaded kernel ROM libraries with versions.

```
UAOS> libs
```

date Display current local date and time. When `ntpd` has run, shows timezone-aware local time. Otherwise falls back to CMOS RTC (UTC).

```
UAOS> date
Saturday 14-Jun-2026 23:12:57 AEST (Australia/Sydney)
```

info [device] Show mounted disks and volumes. With an argument, show details for a specific device.

```
UAOS> info
UAOS> info RAM:
UAOS> info DH0:
```

disks List detected block devices (whole disks only).

```
UAOS> disks
```

which <cmd> Locate a command: reports if it is a shell built-in, a C: binary, or found in the PATH.

```
UAOS> which dir
c:dir
UAOS> which cd
cd is a shell built-in command
```

Filesystem Commands

dir [**path**] List directory contents. If no path is given, lists the current directory.

```
UAOS> dir
UAOS> dir C:
UAOS> dir RAM:T
```

makedir <path> Create a new directory.

```
UAOS> makedir RAM:MyDir
```

delete <path> Delete a file or an **empty** directory.

```
UAOS> delete RAM:T/old.txt
```

type <file> Print the contents of a file to the shell.

```
UAOS> type S:Startup-Sequence
```

copy <src> <dst> Copy a file.

```
UAOS> copy S:Startup-Sequence RAM:T/backup.txt
```

rename <from> <to> Rename or move a file or directory.

```
UAOS> rename RAM:T/old.txt RAM:T/new.txt
```

filenote <file> <comment> Set or update a file comment.

```
UAOS> filenote RAM:T/notes.txt "My notes"
```

relabel <device> <name> Rename a volume.

```
UAOS> relabel RAM: TempRAM
```

protect [+r|-r][+h|-h] <path> Set file protection attributes.

Flag	Meaning
+r	Set Read-Only
-r	Clear Read-Only
+h	Set Hidden
-h	Clear Hidden

```
UAOS> protect +r RAM:important.txt
```

```
UAOS> protect -h +r RAM:config.dat
```

attr <path> Display file or directory attributes.

```
UAOS> attr RAM:important.txt
Attributes: Read-Only
```

list [**path**] List directory contents with detailed information (similar to **dir** with size and attributes).

```
UAOS> list
UAOS> list C:
```

search <pattern> [file] Search files for a pattern. Supports recursive search (ALL/S), a starting directory (FROM/K), file pattern (FILEPAT/K), and case-insensitive matching (CI/S).

```
UAOS> search error S:Startup-Sequence
UAOS> search "TODO" RAM: ALL
```

sort [file] [options] Sort the lines of a file. Options include COL/K/N (column), CASE/S (case-sensitive), NUMERIC/S (numeric sort).

```
UAOS> sort RAM:T/names.txt
UAOS> sort RAM:T/data.txt NUMERIC COL=2
```

join <file1> <file2> Join two files by a common key.

```
UAOS> join RAM:T/a.txt RAM:T/b.txt
```

echo <text> Print text to the shell. Supports variable expansion with \$var.

```
UAOS> echo "Hello World"
UAOS> echo "User is $User"
```

Ring-3 Userspace Commands

These commands are built from `system/userspace/` as native x86-64 ELF64 binaries. They run in user mode and use the INT 0x80 syscall ABI.

pwd Print the current working directory.

```
UAOS> pwd
RAM:
```

file <path>... Identify the format of one or more files from magic numbers.

```
UAOS> file C:dir C:loadwb S:Startup-Sequence
C:dir: UAOS x86-64 ELF64 binary
C:loadwb: UAOS native shell command
S:Startup-Sequence: ASCII text
```

strings <path>... [-n minlen] Scan files for printable character sequences.

```
UAOS> strings C:pwd
UAOS> strings C:lha -n 6
```

find [path] [-name pattern] [-type f|d] Recursively walk a directory tree and list matching path entries.

```
UAOS> find RAM:
RAM:MyDir
RAM:MyDir/notes.txt
UAOS> find RAM: -name *.txt
RAM:MyDir/notes.txt
UAOS> find -type d
RAM:MyDir
```

hello Print a simple greeting (userspace validation utility).

```
UAOS> hello
Hello from UAOS userspace!
```

Guide Open the AmigaGuide help viewer (Tools:Guide).

```
UAOS> Guide
```

Disk Management

fdisk <device> Interactive partition table editor. Enter interactive mode with commands **m** (help), **n** (new), **d** (delete), **p** (print), **w** (write), **q** (quit).

```
UAOS> fdisk virtio0
UAOS> fdisk -l          ; list available disks
```

format <device> [filesystem] Format a partition (not a whole disk). Supports **fat32**.

```
UAOS> format virtio01 fat32
UAOS> format Device=DH0: Name=Workbench FFS
```

Networking Commands

ifconfig [dhcp | <ip> <gateway>] Show or configure network settings. /24 netmask is assumed.

```
UAOS> ifconfig          ; show config
UAOS> ifconfig 192.168.1.10 192.168.1.1
UAOS> ifconfig dhcp      ; run DHCP discovery
```

ping <host> [count] Send ICMP echo requests. Default count is 4, max is 64.

```
UAOS> ping 8.8.8.8
UAOS> ping google.com 8
```

route Display the kernel routing table and ARP cache.

```
UAOS> route
```

nslookup <hostname> [server] Resolve a hostname to an IP address using the configured DNS server.

```
UAOS> nslookup google.com
UAOS> nslookup google.com 8.8.8.8
```

ntpd [server] One-shot NTP time synchronisation. Reads **S:ntp.conf** and **S:timezone.conf**.

```
UAOS> ntpd
UAOS> ntpd pool.ntp.org
```

netstart Initialise the TCP/IP stack, read **S:net.conf**, and start **bsdsocket.library**.

```
UAOS> netstart
```

netstop Shut down the network stack and release the DHCP lease if used.

```
UAOS> netstop
```

netinfo Open the network information window.

UAOS> netinfo

Text Processing

grep [-i] <pattern> <file> Search file contents for a pattern. Use -i for case-insensitive matching.

UAOS> grep error S:Startup-Sequence

UAOS> grep -i amiga RAM:notes.txt

more <file> Paginate file output one screen at a time.

Key	Action
Space / Page-Down	Advance one page
Enter	Advance one line
q / Escape	Quit

UAOS> more S:Startup-Sequence

Desktop & Tools

clear Clear the shell window and history buffer.

UAOS> clear

reboot Warm reboot via the keyboard controller.

UAOS> reboot

pointer Open the Pointer Preferences window.

UAOS> pointer

calculator Open the Calculator window.

UAOS> calculator

clock Open the Clock window.

UAOS> clock

loadwb Launch the Workbench desktop.

UAOS> loadwb

vim <file> Open the inline text editor in the shell window.

UAOS> vim RAM:T/notes.txt

newcli / newshell Open a new shell window. The optional **from <script>** runs a startup script in the newly opened shell (resolved relative to the invoking shell's current directory), mirroring the S:Startup-Sequence mechanism used at boot.

```
UAOS> newcli
UAOS> newcli from S:MySetup
```

Amiga Compatibility

run <program> [args] Execute an embedded Amiga M68k binary from the ROM registry.

```
UAOS> run Calculator
```

assign [name: target] Create or list AmigaDOS assigns. With no arguments, lists current assigns.

```
UAOS> assign                ; list assigns
UAOS> assign C: Workbench:C
UAOS> assign ENV: RAM:ENV
```

execute <script> Run a script file line by line with full flow-control support.

```
UAOS> execute S:Startup-Sequence
UAOS> execute RAM:T/myscript
```

Process and Task Commands

ps List running tasks.

```
UAOS> ps
```

jobs List background jobs.

```
UAOS> jobs
```

wait Wait for background jobs to complete.

```
UAOS> wait
```

runback <command> [args] Run a command line as a background job, returning to the prompt immediately. Equivalent to appending & to the command.

```
UAOS> runback dir Workbench:
```

changetaskpri <priority> [task] Change the priority of a task.

```
UAOS> changetaskpri 5 Shell
```

Shell and Script Commands

ask <prompt> Display a prompt and read a line of user input.

```
UAOS> ask "Continue? "
```

prompt <string> Set a custom shell prompt. Pass an empty string to reset.

UAOS> prompt "\$cwd> "

why Show the return code of the last command.

UAOS> why

failat <n> Set the failure threshold (default 10). Commands returning a code greater than this are considered failures.

UAOS> failat 5

quit [rc] Exit the current script with an optional return code.

UAOS> quit 0

endcli Close the current shell window.

UAOS> endcli

resident Manage resident commands kept in memory.

UAOS> resident C:dir

UAOS> resident

resload <command> [pure|remove] Shorthand for **resident** <cmd> — loads a command into the resident list so later invocations skip disk access. With no arguments, lists resident commands.

UAOS> resload dir

UAOS> resload dir pure

Environment Commands

getenv <name> Read the value of a global environment variable.

UAOS> getenv Language

unset <name> Remove a variable (same as the built-in **unset**).

UAOS> unset greeting

Disk Maintenance Commands

diskchange <device> Notify the system that a disk has changed.

UAOS> diskchange DH0:

addbuffers <device> <n> Add disk buffers to a device.

UAOS> addbuffers DH0: 10

install <device> [NOBOOT] Install a boot block on a device.

UAOS> install DH0:

Dialog Commands

requestchoice <title> <body> <buttons...> Show a choice dialog and return the selected button index.

UAOS> requestchoice "Confirm" "Delete file?" "Yes" "No"

requestfile [TITLE/K] [DRAWER/K] [FILE/K] [PATTERN/K] [PUBSCREEN/K] Show a file requester dialog.

UAOS> requestfile TITLE="Open" DRAWER=RAM:

Utility Commands

status [FULL/S] [TCB/S] [CLI/S] Show system status.

UAOS> status

UAOS> status FULL

stack Show stack usage.

UAOS> stack

avail [BYTES] [K] Show available system memory, queried live from the kernel memory API (SYSCALL_MEMINFO). Reports the x86-64 userspace heap (total/used/free), the emulated M68k guest RAM slot usage, and a scheduler task summary. BYTES prints raw byte counts; K (the default) prints kilobytes/megabytes.

UAOS> avail

UAOS> avail BYTES

Scripting & Flow Control

UAOS scripts are plain text files executed line-by-line via the **execute** command (or the **execute** shell built-in). Lines beginning with **;** or ***** are treated as comments.

Single-Line IF

IF <condition> THEN <command>

Examples:

IF EXISTS S:timezone.conf THEN echo "Config found"

IF \$count EQ 5 THEN echo "Five"

IF NOT EXISTS RAM:lock THEN echo "No lock file"

Multi-Line IF / ELSE / ENDIF

IF <condition>

 <commands...>

ELSE

 <commands...>

ENDIF

Example:

```

IF EXISTS S:timezone.conf
    echo "Timezone configured"
    C:ntpd
ELSE
    echo "No timezone config"
ENDIF

```

FOR / ENDFOR

```

FOR <var> = <start> TO <end> [STEP <step>]
    <commands...>
ENDFOR

```

Examples:

```

FOR i = 1 TO 3
    echo "Loading module $i..."
ENDFOR

```

```

FOR count = 10 TO 0 STEP -2
    echo "Countdown: $count"
ENDFOR

```

Conditions

Condition	Meaning
EXISTS <file>	True if file or directory exists
<a> EQ 	Strings are equal
<a> NE 	Strings are not equal
NOT <condition>	Logical negation

Note: Conditions are evaluated left-to-right. Variable references with **\$var** are expanded before the condition is tested.

Script Arguments (.key and <argname>)

A script can declare named template arguments with a **.key** directive, then reference them with **<argname>**:

```

.key name/A,age/N
echo Hello <name>, you are <age> years old

```

Run it with **execute S:myscript Alice 42** and **<name>** expands to **Alice**, **<age>** to **42**.

The **.key** line uses the same AmigaDOS template syntax as command templates. Qualifiers are fully honoured by **execute** when binding arguments:

Qualifier	Behaviour in scripts
/A	Required — execute reports an error if the argument is missing
/K	Keyword — must be passed as name=value or name value (order-independent)
/S	Switch — <argname> expands to 1 if present, empty if absent
/N	Numeric — value is expected to be a number
/M	Multiple — <argname> expands to all collected values joined by spaces
/F	Free-form — absorbs all remaining tokens into one string

With /K keyword args, the order of arguments on the command line doesn't matter:

```
.key name/A/K,age/K/N
echo Hello <name>, you are <age> years old
```

All of these are equivalent:

```
execute S:myscript name=Alice age=42
execute S:myscript age=42 name=Alice
execute S:myscript name Alice age 42
```

When a `.key` template is present, `$1..$9` are set in **template-item order** (matching the order names appear in the `.key` line), not raw token order. This ensures `<argname>` (which maps names to positions) always resolves to the correct value. When there is no `.key` declaration, `$1..$9` fall back to raw positional tokens as before.

- `<argname>` is only substituted when the name matches a declared key; otherwise `<` is left untouched (so input redirection like `cmd < file` still works).
- Outside a script with a `.key` declaration, `<...>` is never treated as an argument reference.
- `$$` always expands to the full raw argument string.
- If template matching fails (e.g. a missing /A arg), `execute` prints a warning and falls back to raw positional assignment so the script still runs.

Backtick Command Substitution

Text enclosed in backticks (``command``) is replaced by the output of running `command`, with a single trailing newline stripped:

```
echo Today is `date`
echo You have `dir RAM: ALL | wc -l` entries
```

Backtick substitution happens during variable expansion, so it works in command lines, `echo` arguments, and IF condition strings. Nested backticks and `$var` references inside the backtick command are expanded as normal.

Environment Variables

Local Variables (`set` / `unset`)

Stored in the shell instance. Lost when the shell closes.

```
Set greeting "Hello"
echo "$greeting World"
```

Global Variables (`setenv` / `unsetenv`)

Stored in ENV: as files. Persist across sessions.

```
SetEnv Language "english"
UnSet Language
```

Pre-Defined Variables

Variable	Source
<code>\$Workbench</code>	Set during startup (then unset)
<code>\$Kickstart</code>	Set during startup (then unset)
Any ENV: file	Readable as <code>\$name</code>

Variable Expansion

Use `$name` or `${name}` (simple prefix expansion is supported). Variables are expanded: - In `echo` arguments - In IF condition strings - In general command lines before execution

In addition to `$name` variables, the shell supports: - `$(expr)` — arithmetic expansion (e.g. `echo $[2+3]`). - `<argname>` — script template argument reference (see Script Arguments). - ``command`` — backtick command substitution (see Backtick Command Substitution).

```
Set user "Alice"
echo "Hello, $user"
IF EXISTS ENV:Language THEN echo "Language is set"
echo "2 + 3 = $[2+3]"
```

File Assigns

AmigaDOS assigns map logical volume names to physical paths. They are essential for Amiga compatibility.

Kernel-Default Assigns

Assign	Target
C:	Workbench:C
S:	Workbench:S
L:	Workbench:L
DEVS:	Workbench:DEVS
LIBS:	Workbench:LIBS
SYS:	Workbench: (boot volume root)

Common Startup Assigns

```
Assign ENV: RAM:ENV
Assign T: RAM:T
Assign Clips: RAM:Clipboards
Assign REXX: S:
Assign PRINTERS: DEVS:Printers
Assign KEYMAPS: DEVS:Keymaps
Assign LOCALE: SYS:Locale
Assign LIBS: SYS:Classes ADD
```

Creating Your Own

```
Assign MyFiles: RAM:MyStuff
```

I/O Redirection

The shell supports three redirection operators:

Operator	Description
<code>> file</code>	Overwrite output to file
<code>>> file</code>	Append output to file

Operator	Description
< file	Read input from file (placeholder; not all commands support it)

```
dir > RAM:listing.txt
echo "Log entry" >> RAM:log.txt
version >NIL:           ; suppress output
```

Note: >NIL: and similar pseudo-assigns are treated as literal paths unless NIL: is explicitly mounted.

Resident Commands

The **resident** command keeps frequently-used binaries in a 256 KB in-memory cache so they do not need to be re-loaded from disk. Up to 16 resident commands can be cached. Resident commands may be native C: binaries, M68k binaries, or scripts.

```
UAOS> resident C:dir           ; make dir resident
UAOS> resident                 ; list resident commands
```

Example Scripts

Basic Startup Script

```
; Ultimate Amiga OS - Startup-Sequence
C:Version >NIL:

Assign >NIL: ENV: RAM:ENV
Assign >NIL: T: RAM:T

SetEnv Language "english"

; Welcome message
echo "Ultimate Amiga OS - Live Workbench Environment"
echo ""

; Demo: conditional execution
IF EXISTS S:timezone.conf
    echo "Timezone configuration present"
ENDIF

; Demo: numeric loop
FOR i = 1 TO 3
    echo "Loading system module $i..."
ENDFOR

; Synchronise system clock
C:ntpd

; Load Workbench desktop
echo ""
echo "Starting Workbench..."
C:LoadWB
```

Backup Script

```
; backup.script - copy important files to RAM:
IF NOT EXISTS RAM:Backup
    mkdir RAM:Backup
ENDIF

copy S:Startup-Sequence RAM:Backup/
copy S:ntp.conf RAM:Backup/
copy S:timezone.conf RAM:Backup/

echo "Backup complete."
```

Network Test Script

```
; nettest.script - verify network connectivity
IF EXISTS ENV:Server
    echo "Testing server $Server..."
    ping $Server 4
ELSE
    echo "SetEnv Server <address> first"
ENDIF
```

Directory Scanner

```
; scan.script - list and count files
echo "Scanning C:..."
dir C: > RAM:dirlog.txt
grep dir RAM:dirlog.txt
echo "Scan saved to RAM:dirlog.txt"
```

Quick Reference Card

Built-Ins

Command	Syntax	Purpose
help	help	List commands
cd	cd [path]	Change directory
alias	alias [name cmd]	Manage aliases
unalias	unalias <name>	Remove alias
set	set [name value]	Local variables
unset	unset <name>	Remove local var
path	path [dirs...]	Search path
setenv	setenv <name> <value>	Global variable
unsetenv	unsetenv <name>	Remove global var
showconfig	showconfig	Hardware info

Native C: Commands

Command	Syntax	Purpose
version	version	OS version
mem	mem	Memory layout

Command	Syntax	Purpose
libs	libs	ROM libraries
date	date	Current date/time
info	info [dev]	Disk/volume info
disks	disks	Block devices
which	which <cmd>	Locate command
dir	dir [path]	List directory
mkdir	mkdir <path>	Create directory
delete	delete <path>	Delete file/dir
type	type <file>	Print file
copy	copy <src> <dst>	Copy file
rename	rename <from> <to>	Rename/move
protect	protect [+<->] [rh] <path>	Set attributes
attr	attr <path>	Show attributes
filenote	filenote <file> <comment>	File comment
relabel	relabel <dev> <name>	Rename volume
echo	echo <text>	Print text
fdisk	fdisk <dev> / fdisk -l	Partition editor
format	format <dev> [fs]	Format partition
diskchange	diskchange <dev>	Disk change
addbuffers	addbuffers <dev> <n>	Add buffers
install	install <dev> [NOBOOT]	Install boot block
ifconfig	ifconfig [dhcp \> ip gw]	Network config
ping	ping <host> [count]	ICMP echo
route	route	Routing/ARP table
nslookup	nslookup <host> [srv]	DNS lookup
ntpd	ntpd [server]	NTP sync
netstart	netstart	Start network
netstop	netstop	Stop network
netinfo	netinfo	Network info window
grep	grep [-i] <pat> <file>	Search file
more	more <file>	Paginated view
list	list [path]	Detailed dir listing
search	search <pat> [file]	Advanced search
sort	sort [file] [options]	Sort lines
join	join <f1> <f2>	Join files
clear	clear	Clear shell
reboot	reboot	Warm reboot
pointer	pointer	Pointer prefs
calculator	calculator	Calculator win
clock	clock	Clock win
loadwb	loadwb	Launch Workbench
vim	vim <file>	Text editor
newcli	newcli / newshell	New shell window
run	run <cmd>	Run command in new CLI
runback	runback <cmd>	Run command in background
assign	assign [name: tgt]	Create/list assigns
execute	execute <script>	Run script file
ask	ask <prompt>	Prompt user
resident	resident	Manage resident commands
resload	resload <cmd>	Load command resident
ps	ps	List tasks
jobs	jobs	List jobs

Command	Syntax	Purpose
wait	wait	Wait for jobs
changetaskpri	changetaskpri <pri> [task]	Change priority
prompt	prompt <string>	Set prompt
stack	stack	Stack usage
why	why	Last return code
failat	failat <n>	Failure threshold
quit	quit [rc]	Exit script
endcli	endcli	Close shell
getenv	getenv <name>	Read variable
status	status [FULL] [TCB] [CLI]	System status
avail	avail [BYTES] [K]	Available memory
requestchoice	requestchoice <t> <btn...>	Choice dialog
requestfile	requestfile [options]	File requester

Ring-3 Userspace Commands

Command	Syntax	Purpose
pwd	pwd	Working directory
find	find [path] [-name pat] [-type f\ d]	Recursive search
file	file <path>...	Identify format
strings	strings <path>... [-n minlen]	Printable strings
hello	hello	Greeting
Guide	Guide	Help viewer

Script Keywords

Keyword	Syntax
IF	IF <cond> THEN <cmd> or block with ELSE / ENDIF
FOR	FOR <v> = <a> TO [STEP <s>] ... ENDFOR
LAB	LAB <name> — label marker (SKIP target)
SKIP	SKIP <label> or SKIP [<n>] [BACK] — jump to label / skip lines
Comments	; comment or * comment